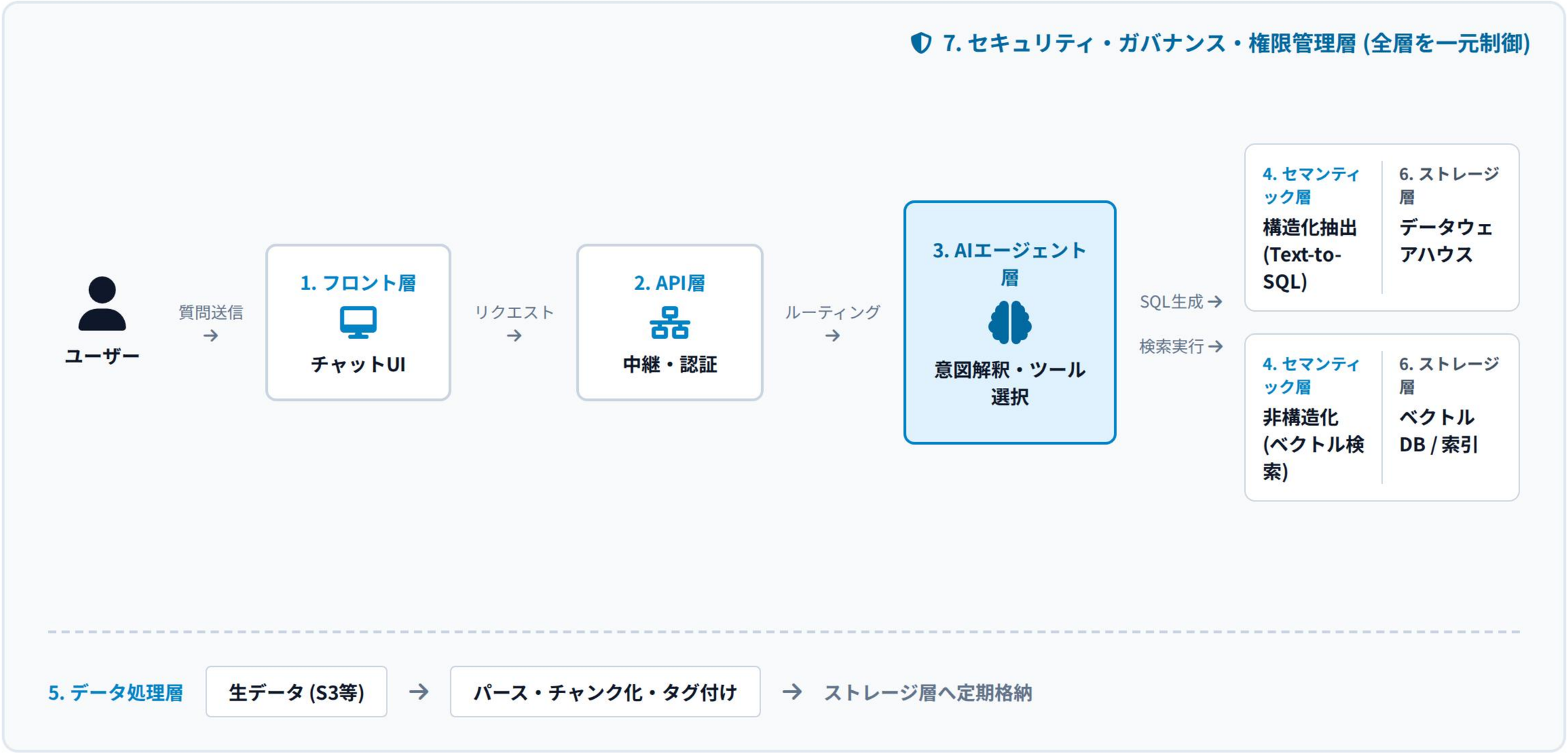
ARCHITECTURE PROPOSAL

構造化・非構造化データを統合する社内RAG基盤アーキテクチャ

Snowflake Cortexを活用したハイブリッド抽出と7つの階層構造

システム構成図：ユーザー入力から回答出力までの流れ



1. フロントエンド層（ユーザーインターフェース）

ユーザーが自然言語でAIエージェントと対話するための画面です。チャットUIだけでなく、過去の会話履歴の表示、参考元ドキュメント（Citation）のプレビュー、抽出された構造化データ（表グラフ）の表示機能などが求められます。

- **フルスクラッチ開発:** React, Vue.js など
- **手軽なUI構築:** Streamlit, Gradio
- **プラットフォーム標準:** Snowflake CoWork（Agents UI）、社内の既存ポータルへの組み込み

2. API / インテグレーション層

フロントエンドからのリクエストを受け付け、AIエージェント層へ渡す「窓口」です。認証トークンの検証、セッション管理、リクエストのルーティングなどを行います。

- **AWS等で構築する場合:** Amazon API Gateway + AWS Lambda, FastAPI, ECSコンテナなど
- **Snowflake完結の場合:** Snowflake API, Cortex Agent REST API

3. AIエージェント・推論層（オーケストレーション層）

ユーザーの質問意図を理解し、「どのデータ（構造化か非構造化か）を見に行くべきか」「どのツールを使うべきか」を自律的に判断して実行計画を立てる、システムの「頭脳」です。

- **主な役割:** ユーザーの曖昧な質問を解釈、ツールの選択（非構造化検索 or SQL生成）、複数ステップの推論（ReActなど）、最終回答の生成。
- **実装例:** AWS Bedrock Agents, LangChain / LangGraph, Snowflake Cortex Agent

4. セマンティック・抽出層（ツール・インターフェース層）

AIエージェントが、実際のデータベースや検索エンジンとやり取りするための「翻訳・抽出レイヤー」です。RAG検証の資料でも最重要視されていた部分です。

- **構造化データ向け（Text-to-SQL）:** ユーザーの質問から正確なSQLを生成するための意味定義（セマンティックモデルやYAML辞書）の層。例: Cortex Analyst、Cube。
- **非構造化データ向け（ベクトル検索）:** 質問文の揺らぎを吸収するクエリ拡張（用語辞書・GLOSSARYによる拡張）や、意図に沿ったドキュメントを抽出する検索エンジン。例: Cortex Search、Amazon OpenSearch、Kendra。

5. データ処理・パイプライン層（インジェスト層）

社内に散らばる生データを、AIが検索・抽出できる形に変換し、継続的に最新状態に保つための裏側の仕組みです。

- **非構造化パイプライン:** PDF等のパース、チャンク分割、メタデータ付与（自動タグ付け）、ベクトル化（Embedding）、差分更新の仕組み。
- **構造化パイプライン:** 業務システムからのデータ抽出（ETL/ELT）、データクレンジング。
- **実装例:** SnowflakeのTask/Stream・PARSE_DOCUMENT、AWS Glue、Step Functionsなど。

6. データ保管・ストレージ層

実際に処理されたデータやベクトル、トランザクションデータを保管する層です。

- **非構造化の保管:** オブジェクトストレージ（Amazon S3, Box, SharePoint等）、ベクトルデータベース（Cortex Searchのインデックス、OpenSearch）。
- **構造化の保管:** データウェアハウス（Snowflake、Amazon Redshiftなど）、リレーショナルDB。

7. セキュリティ・ガバナンス・権限管理層（横断レイヤー）

システム全体を縦に貫く重要な層です。エンタープライズ用途では「このユーザーはこの部署の経費データと、この機密指定ドキュメントしか見られない」といった制御が必須になります。

- **認証・認可:** Azure AD (Entra ID) やOktaなどとのSSO連携。
- **データアクセス制御:** SnowflakeのRBAC（ロールベースアクセス制御）や行アクセスポリシー、AWS IAM。アプリケーション側でのメタデータフィルタリング（属性フィルタ）。

ハイブリッド抽出AIエージェント：開発作業と注意点

必要な開発作業

- **構造化データ抽出：**セマンティックモデル(YAML等)の定義、Text-to-SQL APIの組み込み。
- **非構造化データ抽出：**ドキュメントのチャンク分割、ベクトル検索エンジンの構築と同期。
- **エージェント設定：**抽出ツールを登録し、「ルールの確認は検索、データ集計はSQLを使う」などのシステムプロンプト（指示文）を作成。

設計上の注意点

- **構造化データのベクトル化禁止：**テーブルをテキスト化すると数値集計や厳密な条件抽出が破綻するため、必ずツールを分離して処理させる。
- **用語辞書によるクリーンなクエリ拡張：**ベクトル汚染を防ぐため、メタ文字列を含まないスペース区切りの単語のみで検索クエリを拡張する。
- **無関係なドキュメントの除外：**属性フィルタやLLMプロンプトの指示により、質問と無関係な手順書等が根拠に混ざるのを防ぐ。

アクセス権限管理：複数環境 vs Snowflake一元管理

複数環境（AWS等）で構築する場合

コンポーネントごとの権限マッピング開発が必要

- Entra IDのグループ情報を、AWS IAM、S3ポリシー、OpenSearch ACL、RDS権限等へ同期する仕組みをそれぞれ開発。
- 検索時にユーザー属性をもとにしたアプリケーション側でのメタデータフィルタリング実装が不可欠。
- 権限漏れや設定ミスのリスクが高く、セキュリティ担保の保守コストが膨大。

Snowflakeで一元管理する場合

設定のみで全層のアクセス制御が完了

- Entra IDとSnowflakeをSCIMプロビジョニング等で連携し、ユーザーとグループ（ロール）を自動同期。
- Snowflake内でRBAC（ロールベースアクセス制御）と行アクセスポリシーを設定するだけで完了。
- 構造化データ（テーブル）も非構造化データ（チャンク）も、同一の権限モデルで安全・一元的に制御可能。

Snowflake採用の最大の強み（複数環境構築との比較）

- **インフラ管理ゼロの自動差分更新パイプライン：**
  AWS等で自前構築する場合、パース・チャンク化・差分検知バッチ・ベクトルDB同期等の開発と運用が重荷になります。SnowflakeならTaskとStreamを設定するだけで、ストレージ上のファイル更新が自動でインデックスまで同期されます。

- **既存BI / セマンティックモデルとのシームレスな統合：**
  外部ツールを繋ぐとBIダッシュボードとAIエージェントのデータ定義が分断しがちです。SnowflakeならDWH上で一元管理されたセマンティックモデル（YAML）を直接参照。ビジネスユーザーが日常的に見ているBIと同じ指標・ロジック（Single Source of Truth）を用いて、AIが大規模データを高速集計できます。

- **用語辞書（マスター）の1ソース多重活用：**
  単一の辞書テーブルを「チャンクの自動タグ付け」「検索時のクエリ拡張」「Text-to-SQLの同義語定義」に直接連動させることができ、社内用語の揺らぎによる検索漏れを強固に防ぎます。

Next Steps

API疎結合による共同検証の開始。